A Crash Course on High Availability

#126: What Is High Availability

NEO KIM

FEB 28, 2026 PAID

Get my system design playbook for FREE on newsletter signup:

Subscribed

- *Share this post & I'll send you some rewards for the referrals.*
- *Block diagrams created using Eraser.*

Why should we care about uptime? And what exactly is high availability?

Downtime is expensive...

A payment system that goes down during peak shopping hours bleeds revenue. A hospital record system that crashes mid-shift puts patients at risk. An app going offline for ten minutes triggers trending hashtags from angry users.

Downtime is never abstract...

It translates into lost money, lost trust, and sometimes even legal penalties. Some of it can never be recovered. High availability started long before the cloud. In the 1960s, defense and finance systems had to run nonstop. Those engineers designed machines that could keep working even when parts failed. When the internet arrived, that same discipline moved online. Banks, retailers, and payment networks learned that a brief outage can erase months of profit.

With the widespread use of technology, the expectation of “always on” has never been greater. Now, uptime is not a luxury but a baseline.

The goal never changed: build systems that keep running when the world shakes.

So failures will happen. No matter how hard we try, we can’t avoid them. Hardware burns out, networks drop packets, software engineers create bugs. High availability (**HA**) is about absorbing those failures behind the scenes…it’s about the service being available regardless of failures.

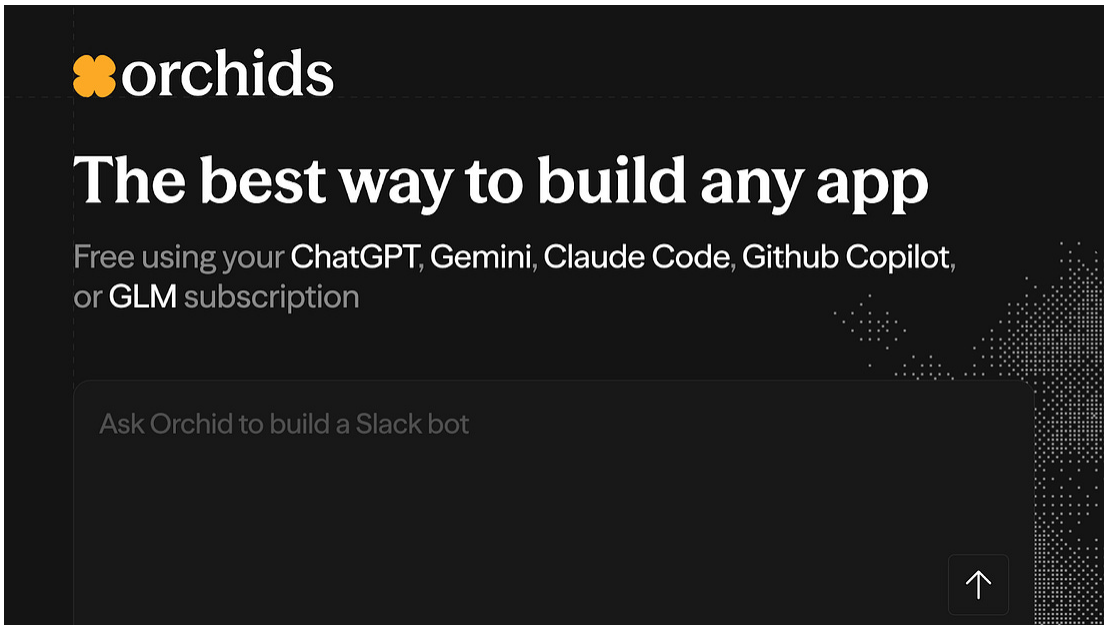
People don’t think about their car tires until one goes flat.

High availability is having a spare one in the trunk. After we replace them, we can drive again. We can’t always control why the tire got flat, but we can carry a spare one. This is the core idea of high availability.

Now let’s put some numbers to it.

Onward.

The best way to build any app (Partner)



Treat your app like an Orchid: A beautiful flower that needs sunlight and a bit of water 🌸

Most "AI builders" make you grow your app in their pot. Same stack. Same limits. Same rules. And on their databases.

Orchids is different:

- It's your build space, set up your way.
- Build anything, Web app, mobile app, Slack bot, Chrome extension, Python script, whatever.
- Bring your own AI subscriptions so you're not paying twice.
- Plug in the database you already use and trust.
- Use any payment infra you want.

Try **Orchids.app** and build it the way you were meant to:

Get Started Today

(Thanks, Orchids, for partnering on this post.)

Use this discount code to get a one time 15% off during checkout: MARCH15

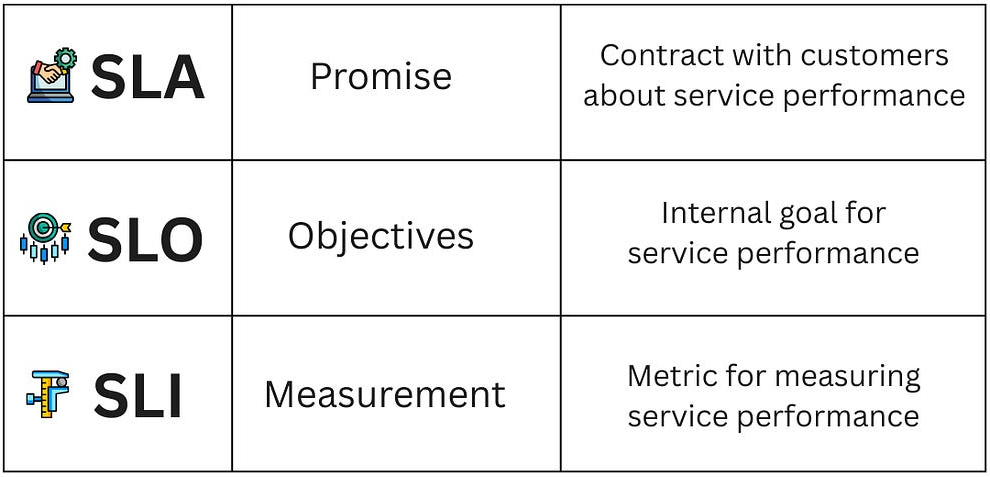
Fundamentals

HA means keeping a system running even when parts of it fail.

The higher the availability, the less impact each individual failure has. To manage HA, engineers use **SLAs, SLOs, and SLIs**. These turn vague ideas like “keep it up and running” into numbers we can measure:

- **Service Level Agreement (SLA):** Contract with customers about service performance.
  - This contract keeps a record of what the service provider promises to deliver
  - There are penalties or costs if the contract is not respected
  - In HA, this is an agreement about how much downtime is acceptable
  - For example, “our app will be online 99.9% of the time. If not, we’ll give your money back.”
- **Service Level Objective (SLO):** Specific internal goal for the service performance.
  - This is the target that internal teams are trying to hit with a desired metric
  - Your SLA is the minimum you promise customers; your SLO is the better performance you target internally
  - For example, if the SLA is 95% uptime, the SLO might be 98% to leave a 3% safety margin
- **Service Level Indicator (SLI):** Metric used to measure service performance.

- Without measuring service performance, we can't know if we're hitting the targets
- These metrics should reflect the SLO and SLA
- For example, "Percentage of failed requests."



SLA vs SLO vs SLI

Let's illustrate it with an example:

- A restaurant promises customers that their food will be delivered within 20 minutes of ordering (SLA).
- The kitchen aims to finish orders in 15 minutes to stay ahead (SLO).
- They track the average order completion time (SLI).

These targets get expressed in "nines of availability".

Availability	Downtime per Year	Downtime per Day
90% (1 nine)	36.5 days	2.4 hours
99% (2 nines)	3.65 days	14.4 minutes
99.9% (3 nines)	8 hours	1.44 minutes
99.99% (4 nines)	52 minutes	8.6 seconds
99.999% (5 nines)	5 minutes	0.86 seconds
99.9999% (6 nines)	32 seconds	86 milliseconds
99.99999% (7 nines)	3.2 seconds	8.6 milliseconds

Nines of availability

One nine means 90% uptime; two nines mean 99%; and so on...

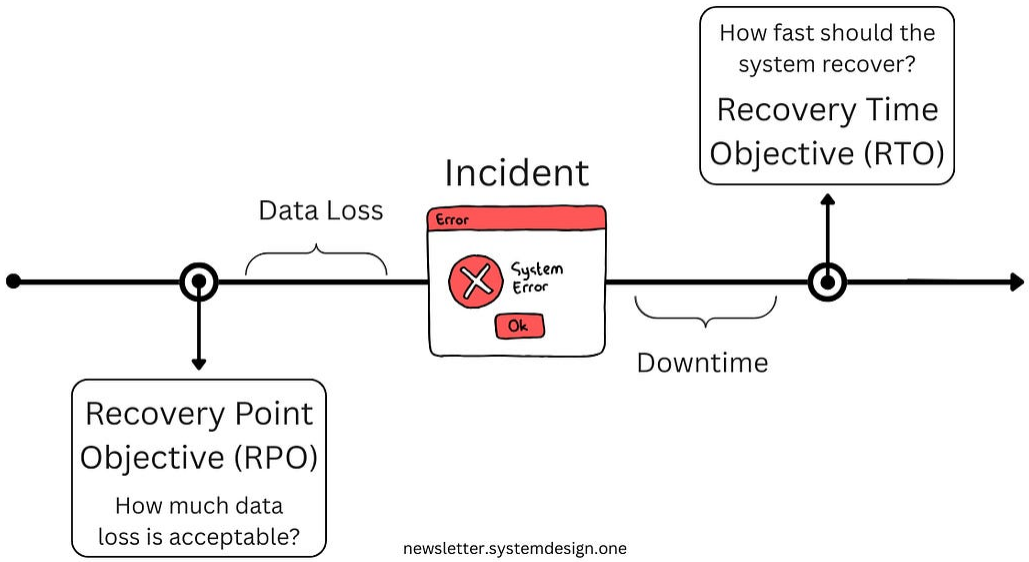
Each extra nine sounds small, but cuts downtime by a ton. For example, 99% uptime allows over 3 days of outage a year, while 99.9% ("three nines") allows only about 8 hours. Every nine added costs more. It needs better hardware, more redundancy, and more monitoring.

The closer you aim for perfect uptime, the more effort and money it takes to maintain it.

When failures occur, recovery metrics help us measure how quickly and effectively we can recover. Here are the most important ones:

- **Recovery Time Objective (RTO)**
  - How fast should the system recover from failure? How long can it be down for?
  - Larger RTO means more downtime is acceptable; smaller RTO means less downtime
  - For example, an RTO of 10 minutes means that the system should be able to recover within 10 minutes of failing

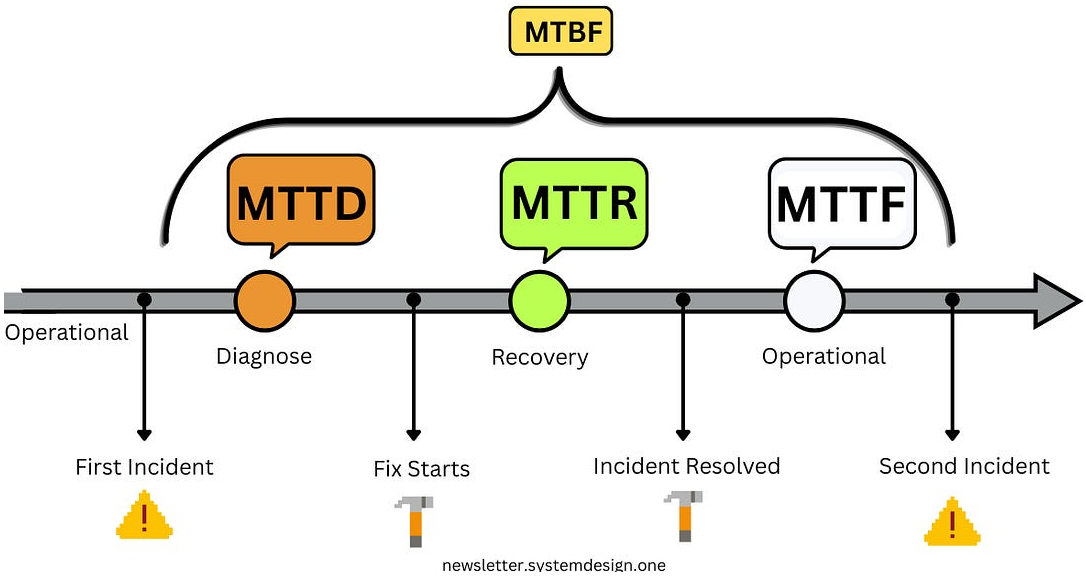
- **Recovery Point Objective (RPO)**
  - To what point in time does the system recover? How much data loss is acceptable?
  - Larger RPO means more data loss, smaller RPO means less
  - For example, an RPO of 5 minutes means 5 minutes of data gets lost



RPO represents gap between last durable recovery point and an incident

- **MTTD (Mean Time to Detect)**
  - This is the mean time needed to notice a failure
  - How long does it usually take for the system or team to detect that something is wrong?
  - Smaller MTTD means faster detection; larger MTTD means slower detection
  - For example, an MTTD of 30 seconds means issues get found half a minute after they occur on average
- **MTTR (Mean Time to Repair)**

    - This is the mean time needed to fix a failure. How long does the system usually take to recover?
    - Larger MTTR means more time to recover, smaller MTTR means less
    - For example, an MTTR of 5 minutes means the failure will take 5 minutes to recover on average
- **MTBF (Mean Time Between Failures)**
    - This is the mean time between two failures. How often does the system usually fail?
    - Larger MTBF means failures happen less often & vice-versa
    - For example, an MTBF of 1h means failures usually happen every hour
- **MTTF (Mean Time to Failure)**
    - This metric is designed for non-recoverable components. How long is the lifespan of this component?
    - This metric differs from MTBF because it lacks a recovery component. The component is alive, and then it crashes without recovery. MTTF is the time between those two points.
    - A larger MTTF means a component has a longer lifespan, and a smaller MTTF means a shorter one
    - For example, an MTTF of 3 years means a component usually lasts for 3 years before becoming unusable



NOTE: MTTF is for non-repairable systems. For repairable systems, it's uptime.

Availability Formula

Availability links uptime & downtime in one line:

```
Availability = MTBF / (MTBF + MTTR)
```

If the system runs for 1000 hours before a 1-hour fix, uptime is 99.9%.

Every extra nine costs more to achieve. Past "three nines," you buy less outage and pay more in redundancy, automation, and testing.

Takeaway:

HA is measurable. Metrics turn abstract goals into clear engineering targets.

What gets measured gets managed.

Ready for the best part?

Reminder: this is a teaser of the subscriber-only post, exclusive to my golden members.

When you upgrade, you'll get:

- **Full access to System Design Case Studies**
- FREE access to (coming) Interview Academy
- **FREE access to (coming) Design, Build, Scale newsletter series**

And more!

Get 10x the results you currently get with 1/10th the time, energy & effort.

Unlock Your Next Level

Patterns

Engineers use HA patterns to make design decisions.

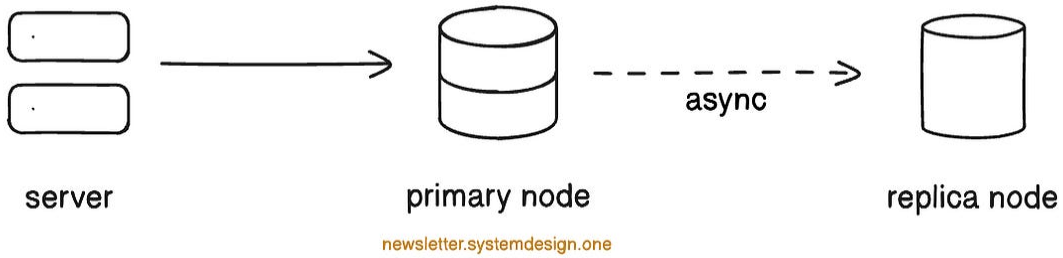
These patterns describe how nodes share the load, replicate data, and relate to one another. Replication is the backbone of HA patterns. It's how systems copy data between themselves and stay in-sync.

It comes in two flavors: asynchronous and synchronous.

Asynchronous replication is fast, but not always in sync.

A node writes data, tells others about it, and immediately moves on. It doesn't wait to see if they got the message. This helps them move on faster and gives them more time to process requests.

The trade-off is that data stays out-of-sync between the nodes for longer...

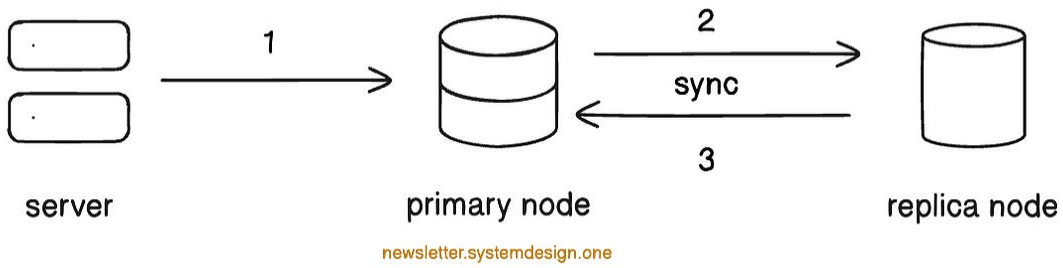


Asynchronous replication flow

Synchronous replication is slower but guarantees data is in sync.

A node writes data and waits until others confirm they copied it before moving on. This makes them wait longer before moving on to process requests.

But the data is in a consistent state.



Write data is commited to primary node only after replica node confirms

These replication types appear in every pattern below...

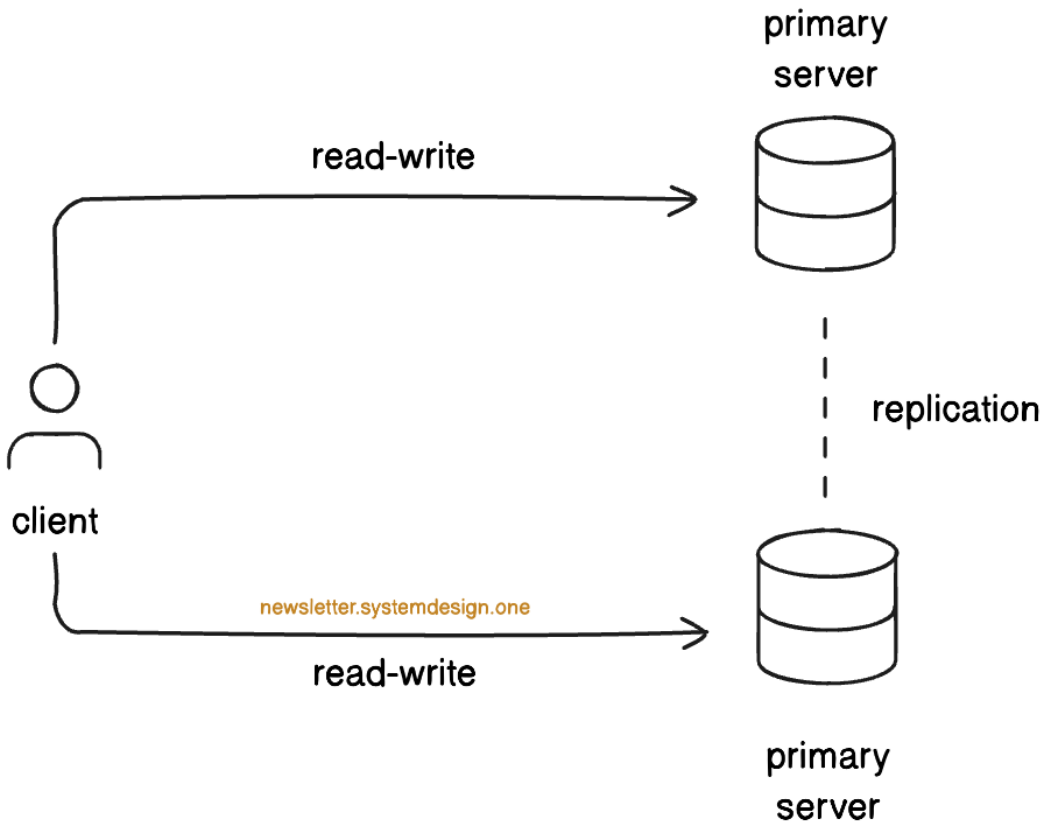
Hot-Hot

Hot-hot means all nodes are alive and active.

They serve traffic at all times, sharing the load. There are no primary and backup nodes--all nodes are equal. This pattern frequently appears on high-frequency trading platforms, in e-commerce checkouts during sales, or in real-time multiplayer games.

It's like a kitchen where all the chefs cook at the same time:

Even if one chef steps away, the others keep working without pausing service. They handle more load by splitting the work. They must coordinate, or the kitchen becomes chaotic.



Hot-Hot pattern

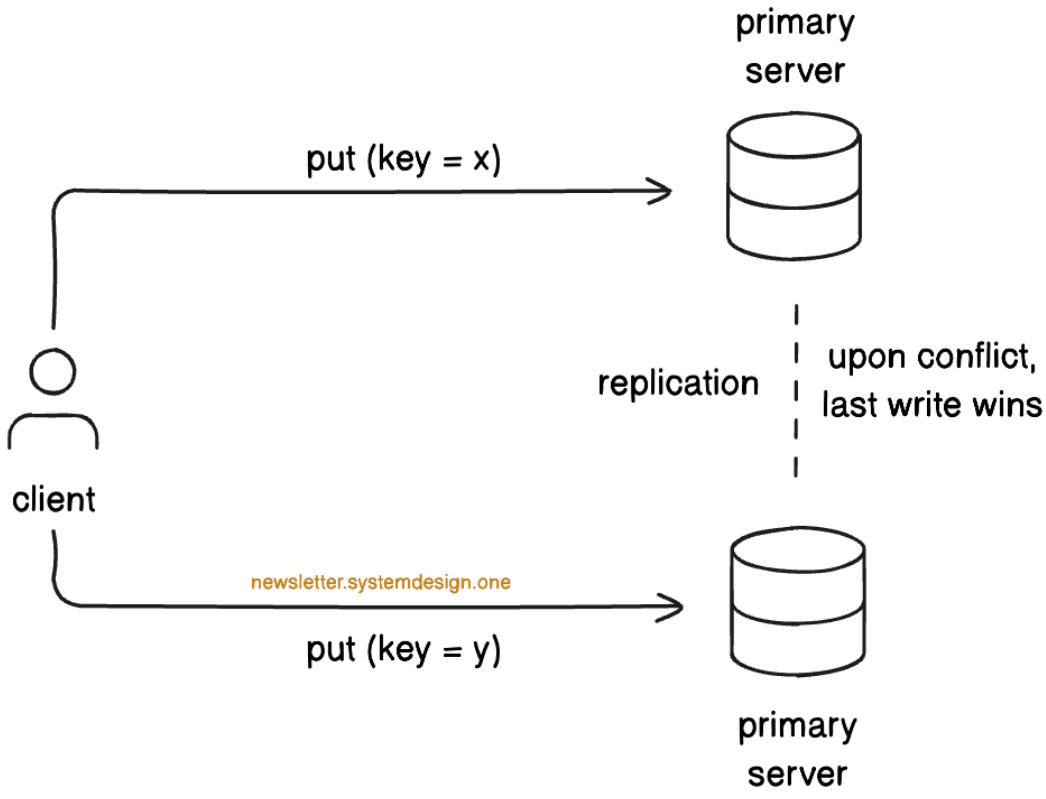
Here's how it works:

- A load balancer spreads requests across nodes, so requests get processed faster
- Replication keeps nodes in-sync, so they all have the latest data
- When a node fails or gets too busy, others easily take over
- Failover is instant because all nodes are up and running; therefore, we call it hot-hot

Failures are invisible to clients. The system absorbs a node loss without interruption.

Yet there are other risks:

- **Write conflicts**
  - When multiple nodes process updates to the same record at once, each node has a different version, with no way to know which is correct
  - Problem: Conflicting changes can overwrite each other or create inconsistent states
  - Solution: Use automatic conflict resolution (e.g., last-write-wins, versioning, or quorum-based consensus)



- **Replication lag**
  - Nodes take time to sync up because data must travel across networks between data centers; a write in New York takes milliseconds to reach Tokyo, so replicas show stale data
  - Problem: We can read outdated information or see different results on different nodes
  - Solution: Use faster replication methods (like dedicated network links between data centers) or design the system to tolerate inconsistencies
- **Increased costs**
  - Keeping many active nodes requires more hardware, energy, and maintenance
  - Problem: Redundancy raises operational expenses

- Solution: Consider other HA patterns (hot-warm, single-leader) for reducing costs

This pattern's upsides are good failover, where users rarely notice failures.

Every replica is always active, which also increases the throughput of the system. But the downsides include cost increases and consistency problems. Plus, replication lag and write conflicts make the system a lot more complex.

Hot-hot maximizes availability but increases complexity. The next pattern trades some availability for simplicity...

Hot-Cold

Hot-cold means one node runs live while another waits quietly as backup.

The primary node serves all traffic. The secondary node stays idle but synced with the latest data. When the main node fails, traffic shifts to the standby.

It's like a generator that stays silent until the lights go out: it does nothing most days, but when it's needed, it saves the operation.

Here's how it works:

- One active node handles all reads & writes
- A passive node receives periodic data syncs from the primary
- When the primary fails, the system switches traffic to the backup node

This pattern is much cheaper than hot-hot because one node is turned off most of the time. This leads to lower operational costs. But it doesn't come for free...

There are a few problematic side effects of this pattern:

- **Idle resources**
  - Even though you're not actively using them, you still need to pay for the "ready-to-use" resources
  - Problem: Standby nodes burn cost without doing work
  - Solution: Use it for background tasks or testing if possible
- **Sync lag**
  - Problem: Backup might be slightly behind primary, causing minor data loss during failover (recent writes not replicated yet)
  - Solution: Reduce sync interval or use continuous replication
- **Slow recovery**
  - Problem: Manual failover adds delay during outages
  - Solution: Automate detection and switchover for faster recovery

Hot-cold is simple and cheap, but has its own tradeoffs...

It suits systems that can tolerate short downtime and partial data replay. The next pattern provides a middle ground between hot-hot & hot-cold.

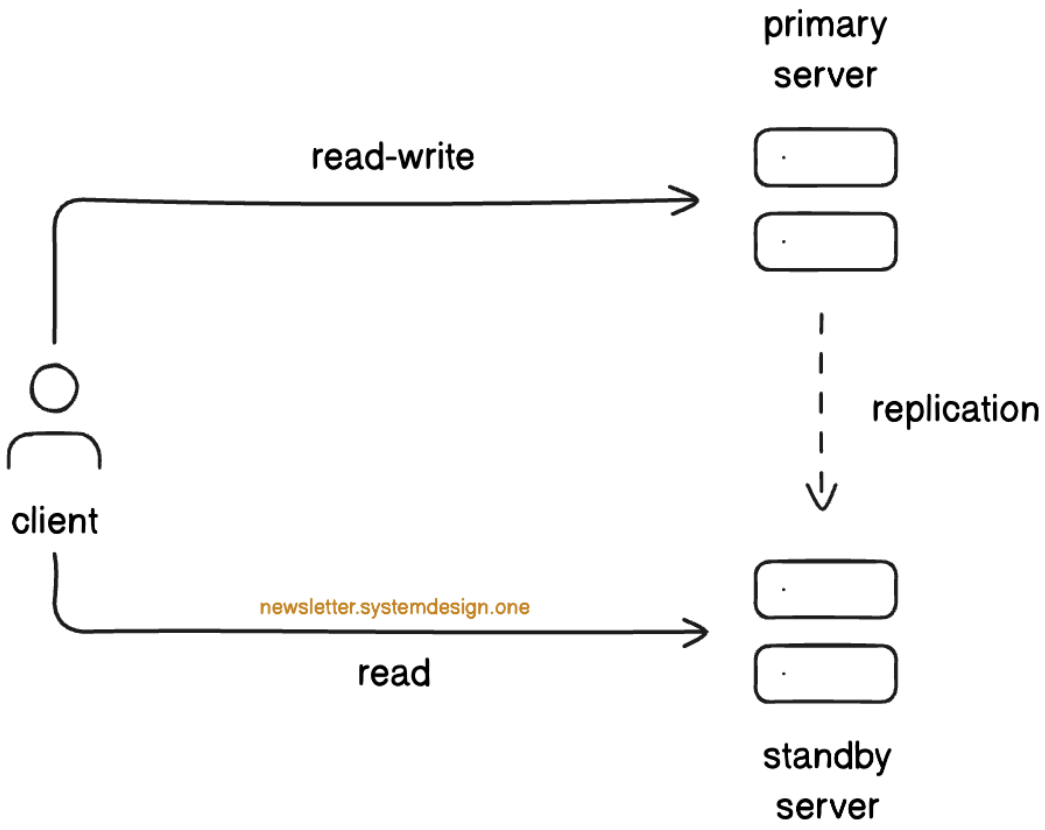
Hot-Warm

Hot-warm means some nodes are active while others are on standby.

Active nodes handle traffic. Standby nodes replicate data and wait to take over if needed. This pattern is frequently used in Software-as-a-Service (SaaS) applications and content delivery platforms. The replicas should be ready, but not enough to keep them on all the time.

It's like a restaurant where chefs cook meals while assistants prepare in the back:

If a chef steps away, an assistant immediately steps in. So customers rarely notice a disruption.



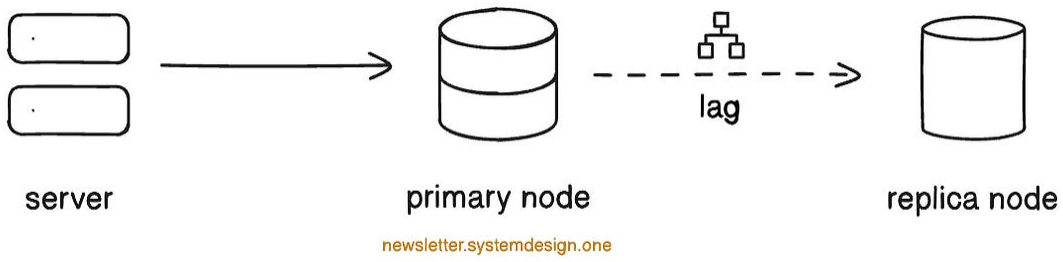
Hot-warm pattern

How it works:

- Active nodes serve all requests under normal conditions
- Standby nodes replicate data from active nodes and remain ready to take over
- If an active node fails or becomes overloaded, a standby gets promoted to active
- Failover is quick, though slower than hot-hot, because of the takeover process

Failures are most often invisible to clients, but there are other risks:

- **Replication lag**
  - Standby nodes may not always be up-to-date with active nodes
  - Problem: During failover, some data may be slightly outdated for a short period
  - Solution: Use synchronous replication for critical data or accept small, tolerable inconsistencies



- **Failover delays**
  - Standby nodes need time to become active
  - Problem: Users can experience a brief pause or slower responses

    - Solution: Monitor replication performance and automate promotion to reduce downtime
- **Underutilized resources**
    - Standby nodes sit idle until needed
    - Problem: We pay hardware and energy costs even for the standby
    - Solution: Scale standby nodes dynamically or balance cost vs availability

This pattern's upsides are lower operational costs than hot-hot and faster failover than hot-cold.

The downsides are failover delays, out-of-sync data, and nodes consuming resources while idle. The next pattern can be combined with the patterns above for even better HA...

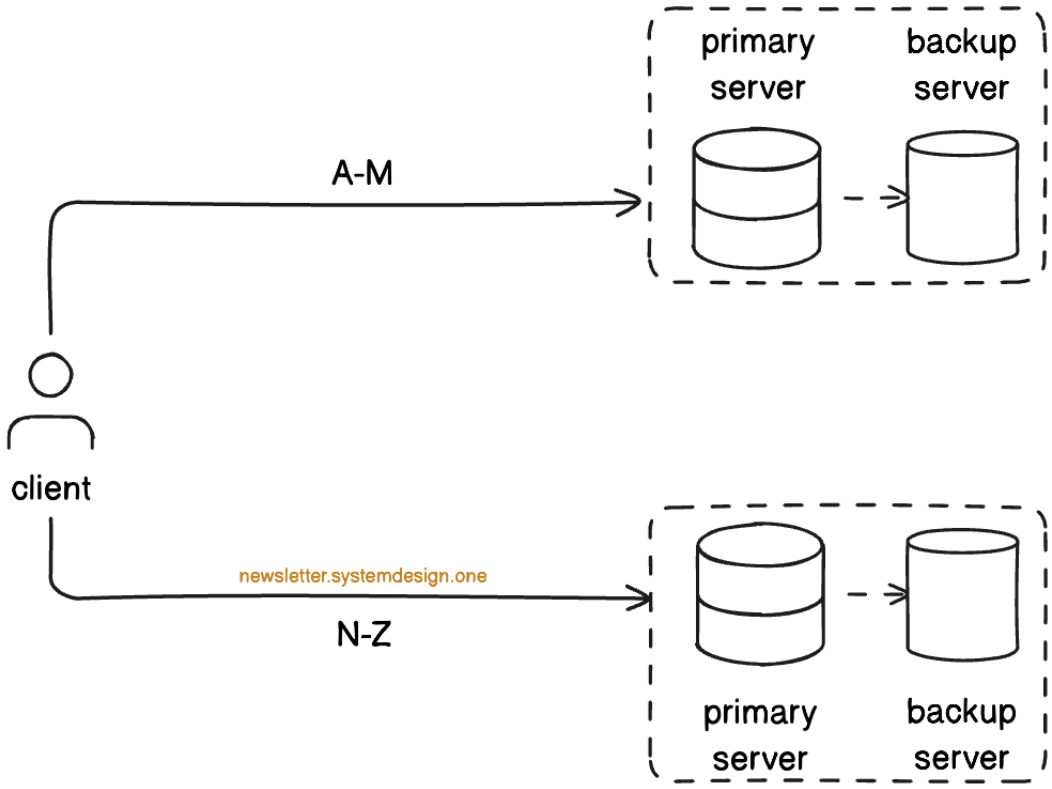
Partitioned Redundancy

Partitioned redundancy means dividing the system into smaller, independent sections.

Each section (or partition) has its own active and backup nodes. If one partition fails, only that slice of traffic is affected. The rest keep working...

It's like a city power grid with backup lines in every district:

If a transformer fails, its backup takes over instantly, and the rest of the city keeps running without noticing. Each district operates independently, so one failure doesn't cascade.



How it works:

- Data and traffic get split into partitions (for example, by user ID or region)
- Each partition has its own active and standby replicas
- Failover happens within the same partition, keeping the blast radius small
- Load balancers or routing layers direct users to the right partition

Partitioned redundancy limits the impact of failures and simplifies scaling.

But it introduces new challenges that must be managed carefully:

- **Hot partitions**

    - When a partition gets most of the traffic, it risks overload; we call it a hot partition
    - Problem: Some partitions may carry more users or data than others, leading to more load
    - Solution: Monitor and rebalance partitions over time
- **Complex management**
    - Problem: More partitions mean more infrastructure to track and coordinate
    - Solution: Use orchestration tools and standardized configs for each partition
- **Cross-partition operations**
    - Problem: Operations needing data from multiple partitions become slower or more complex
    - Solution: Design APIs to minimize cross-partition calls or use aggregation layers

Partitioned redundancy improves isolation, limits impact, and enables per-segment scaling. It's the foundation for large distributed systems where full-system failover is very costly or slow.

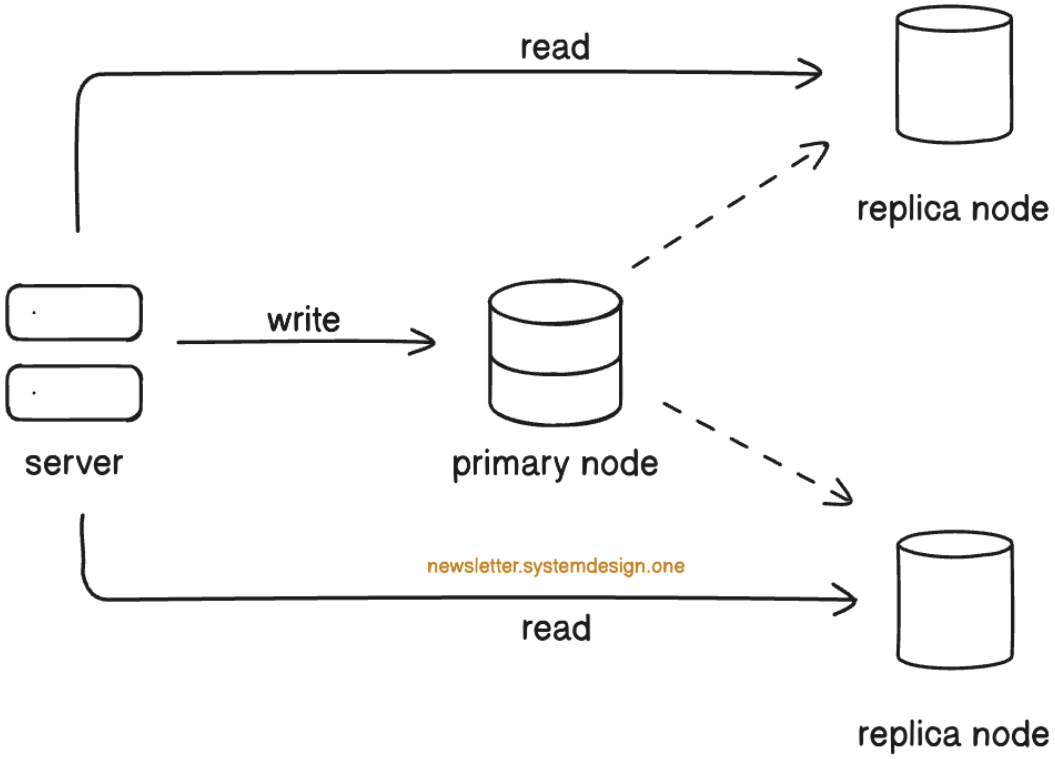
Single-Leader

Single-leader means one primary node handles all writes, while replicas handle read requests.

The leader coordinates updates to ensure data consistency, while followers replicate changes. This pattern is often used in databases and financial systems. Great for when consistency is more important than distributing write load. Writes go through the primary; reads go through the replicas.

It's like a checkout counter at a small store with only one cashier:

Customers can still view items on the shelves, but every payment must be made at the cashier. If the cashier is gone, nobody can buy anything until another cashier takes over. But people can still browse the store.



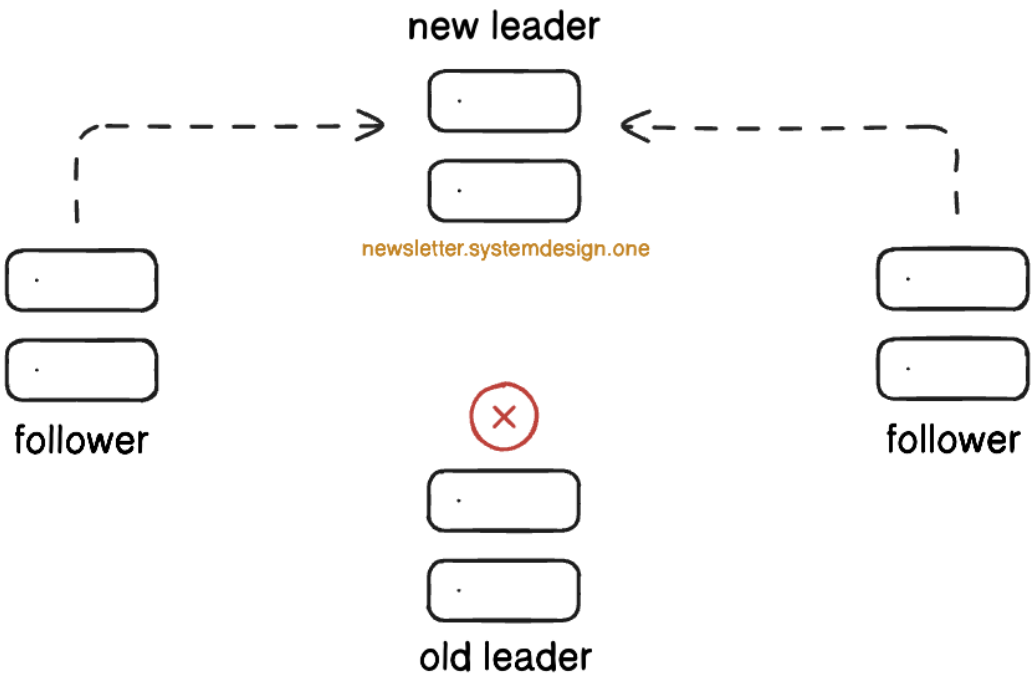
Single-leader pattern

How it works:

- The leader node processes all write requests
- Replicas receive updates from the leader and handle read requests
- If the leader fails, a leader election protocol (e.g., Raft or Paxos) selects a new leader
- Leader election is when nodes decide on a new leader, based on some algorithm
- Failover gets paused during the leader election

Failures are mostly invisible to clients, but there are other risks:

- **Leader failure**
  - If the leader goes down, writes cannot proceed until a new leader gets elected
  - Problem: Write operations pause, causing a temporary service interruption
  - Solution: Monitor the leader and automate election processes to reduce downtime



- **Replication lag**
  - Replicas may be a little behind the leader
  - Problem: Read requests can return outdated data
  - Solution: Tune replication frequency or use synchronous replication for critical data
- **Read-after-write consistency**

    - Users expect to see what they just saved, but replicas may not have the latest data
    - Problem: Inconsistent behavior confuses users and leads to a bad experience
    - Solution: Send reads from the same client session to the leader until replication is over
- **Write bottleneck**
    - All writes pass through a single node
    - Problem: Under high load, the leader can become a performance bottleneck
    - Solution: Optimize leader performance, partition data, or consider sharding to distribute the load

This pattern's upsides are no write conflicts and predictable data consistency.

The downsides are write bottlenecks, failover pauses during elections, and read replica lag.

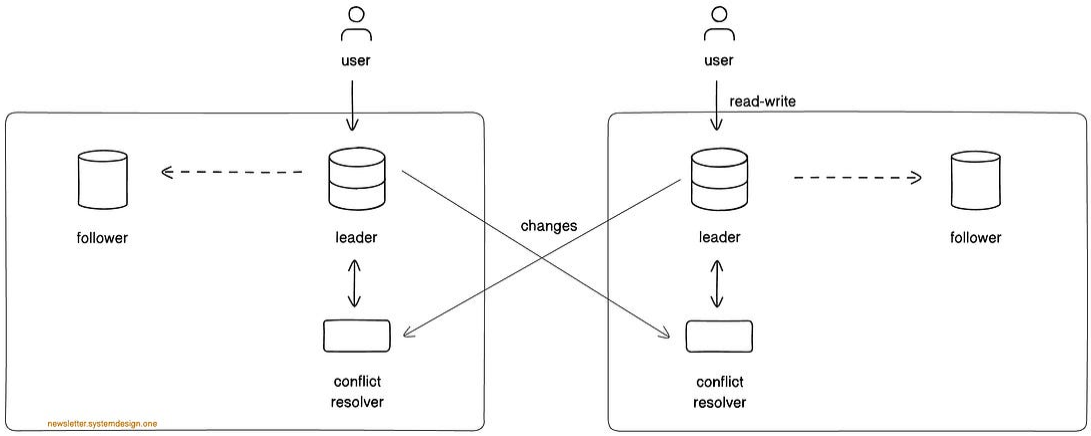
Multi-Leader

Multi-leader means more than one primary node can handle writes at the same time.

Each one shares updates with the others so all maintain a consistent state. If one fails, the others keep working. This is useful when systems run in many places around the world because there's a primary node near every geographic location.

It's like several store cashiers working at once:

Each one can check out customers, and they tell each other what got sold, so the inventory is up to date.



Multi-leader replication

How it works:

- Every primary node (leader) can accept writes
- Follower nodes replicate the leaders, just like in the single-leader pattern
- Each one sends the changes to the others
- If one fails, others keep working without waiting

The upsides are HA and regional write access without a single point of failure.

But there are downsides as well:

- **Conflicting writes**
  - Two leaders can change the same data to different values at the same time
  - Problem: Data can diverge or overwrite itself
  - Solution: Use automatic conflict resolution (e.g., last-write-wins, versioning, or quorum-based consensus)
- **Replication lag**
  - Updates take time to reach other leaders, especially due to geographic distribution
  - Problem: Users in different regions can see different versions of the same data
  - Solution: Speed up cross-leader sync or design for eventual consistency
- **Operational complexity**
  - Coordinating multiple leaders is harder to monitor and debug
  - Problem: More moving parts increase the chance of replication or conflict errors
  - Solution: Automate replication monitoring and keep configurations uniform across leaders

This pattern is best for systems spread across regions that need local write access.

Think of global chat apps or online stores with users in different countries. It works well when a short data-sync delay is acceptable.

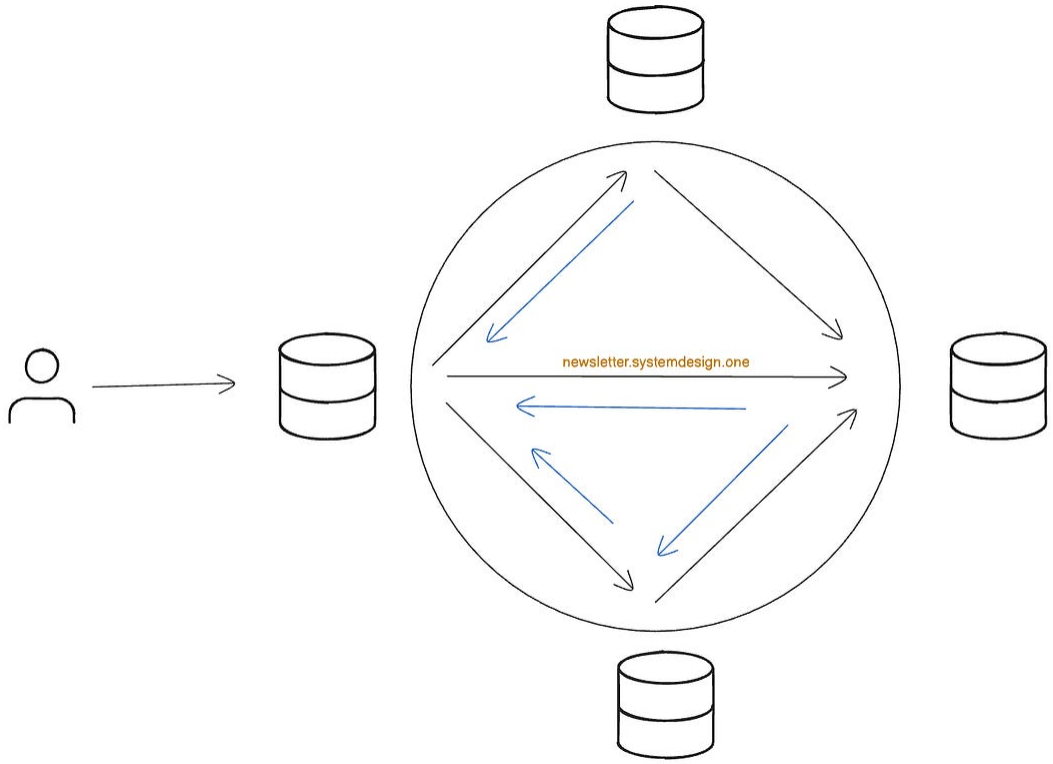
Leaderless

Leaderless is a mode in which all nodes accept writes and work together to replicate data.

There is no single leader controlling the updates. This pattern is often used in distributed databases and large-scale messaging systems. and platforms that must remain available even when nodes fail.

It's like a group of delivery drivers where any driver can pick up and deliver packages:

Each driver keeps a record of what they delivered. Drivers check with each other to make sure nothing gets lost or duplicated.



Leaderless pattern

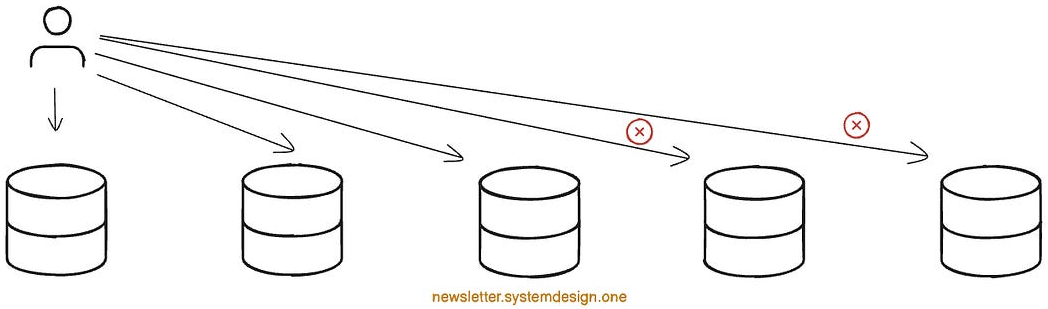
How it works:

- Any node can accept write requests from clients
- Nodes replicate data to each other to stay in sync
- Conflicts happen all the time and need to get resolved

- A quorum system ensures that a majority of nodes agree on the data

Failures are mostly invisible to clients, but there are other risks:

- **Quorum delays**
  - The system waits for agreement from a majority of nodes
  - Problem: Writes can be slower during network issues or node failures
  - Solution: Tune quorum requirements based on criticality and network reliability



Quorum consensus

- **Conflict between writes**
  - Two nodes might accept conflicting updates at the same time
  - Problem: Data can be inconsistent across nodes for a brief time
  - Solution: Use automated conflict resolution strategies, such as versioning or last-write-wins
- **Increased complexity**
  - More nodes writing makes the system harder to monitor and manage

- Problem: Operational mistakes can propagate if conflicts are not handled well
- Solution: Use robust monitoring, logging, and automated recovery processes

This pattern's upsides are high availability and no single point of failure.

The downsides are inconsistencies, slower writes sometimes, and higher operational complexity.

***Takeaway*:**

Each HA pattern offers a way to keep systems running despite failures.

Trade-offs are in speed, cost, and data consistency. The right choice depends on how critical uptime is, how much delay or inconsistency is acceptable, and how much complexity the system can handle.

Patterns define the architecture, but the techniques make high availability work in practice.

Techniques

Extra nodes or backups alone do not stop failures from affecting users. Systems need to detect problems, move traffic, and keep services running under stress.

It's like a city's traffic system:

Roads get blocked, accidents happen, and rush hour slows everything down. Traffic lights and detours keep cars moving. HA techniques do the same job. They spot failing nodes, reroute requests, and keep the system alive even when parts fail.

Load balancing spreads requests across all active nodes...

It prevents any single node from getting overloaded, avoiding a single point of failure. Modern load balancers can detect when a node slows down or fails, then route traffic elsewhere. It happens before users even notice. This balancing also smooths out sudden spikes, e.g., a sale event or breaking news.

Compute and storage separation keeps work and persistence independent...

Compute nodes handle requests and processing. Storage nodes handle data access. This allows you to scale things independently. If traffic spikes, you can add more compute nodes without changing the storage layer. If a compute node fails, a new one can take its place while the storage works without interruption.

Failover & recovery ensure backups take over when the main system is down...

Failover and recovery mechanisms constantly monitor the health of every node. When one goes down, the system switches traffic to a healthy backup. Ideally, this happens automatically and fast enough that users never notice. Recovery also ensures data remains consistent: in databases, writes need to replay on a standby node, so nothing is lost.

Geographic distribution puts nodes in different physical locations...

A single availability zone (**AZ**) can fail because of a power cut, network outage, or flood. By running in different AZs, the system quickly recovers from local problems. Multi-region setups go further, surviving regional disasters like earthquakes or major power failures. This also improves performance because servers closer to users respond faster.

Monitoring and automation track system performance, detect anomalies, and trigger recovery steps...

This ensures small issues do not turn into larger outages. Alerts allow engineers to respond fast when human intervention is needed.

These techniques transform redundancy from passive backup into active protection. Systems continue operating under stress, maintain consistent data, and keep users satisfied.

Takeaway:

Techniques are the backbone of high availability...

They detect failures, reroute traffic, maintain data consistency, and keep systems running. Without them, HA patterns alone cannot guarantee reliable service.

Advanced Techniques

Advanced techniques help systems handle stress, overload, and unexpected failures.

They go beyond just having failovers, keeping the system running when traffic spikes or parts fail. They reduce problems and keep services available.

Rate limiting controls how many requests the system accepts at once...

Think of it as a bouncer at a crowded club:

Only a certain number of people get in, and the rest wait outside. This prevents flooding nodes when too many users try to log in or send requests simultaneously. We don't let everything pile in and collapse the system.

Extra requests get slowed, delayed, or gracefully rejected.

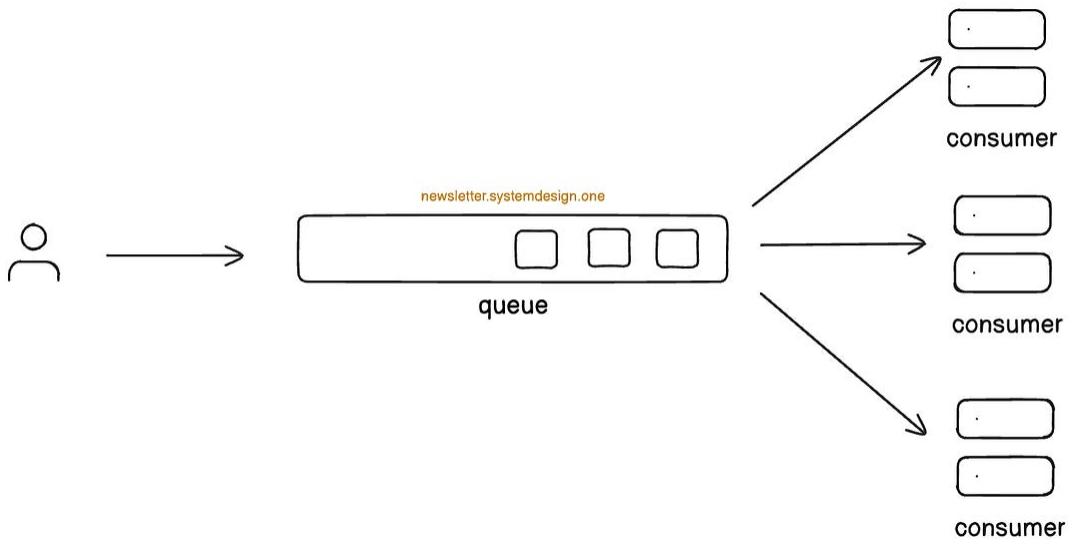
Service degradation turns off non-priority features when the system is busy...

The system prioritizes core functionality over extras. For example, a video app can pause recommendations and thumbnails, but play the videos. This ensures the user's main goal is never blocked, even if supporting features disappear. When done well, users barely notice because the essential service remains available.

Queuing holds requests in line when the system gets overloaded...

Instead of hammering nodes until they crash, requests wait their turn. Think of it like a checkout line at a supermarket:

Even during a rush, each customer gets served in order without breaking the register. Queues absorb traffic spikes and smooth out the load. They also prevent sudden bursts from overwhelming resources. They add latency, but they keep the system stable under pressure.



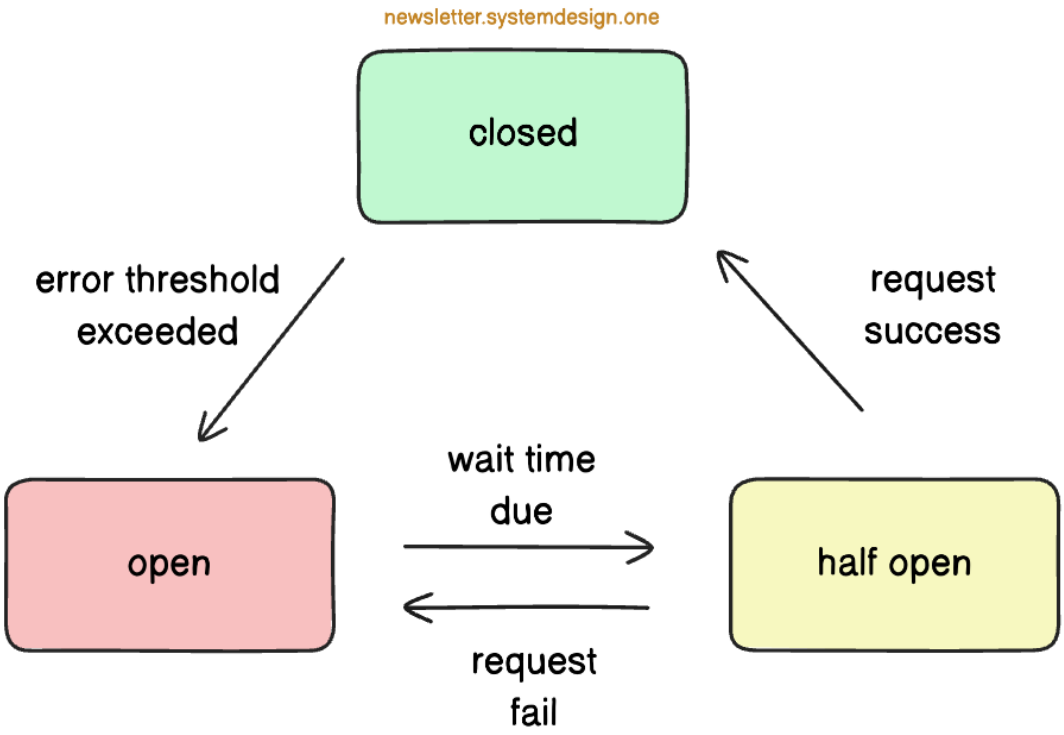
How Queueing Works

Circuit breaking stops requests from going to parts of the system that are failing...

If one service is slow or down, the circuit breaker trips and blocks future calls. This way, the failures don't cascade.

It's like shutting off a faulty electrical circuit before it burns down the house:

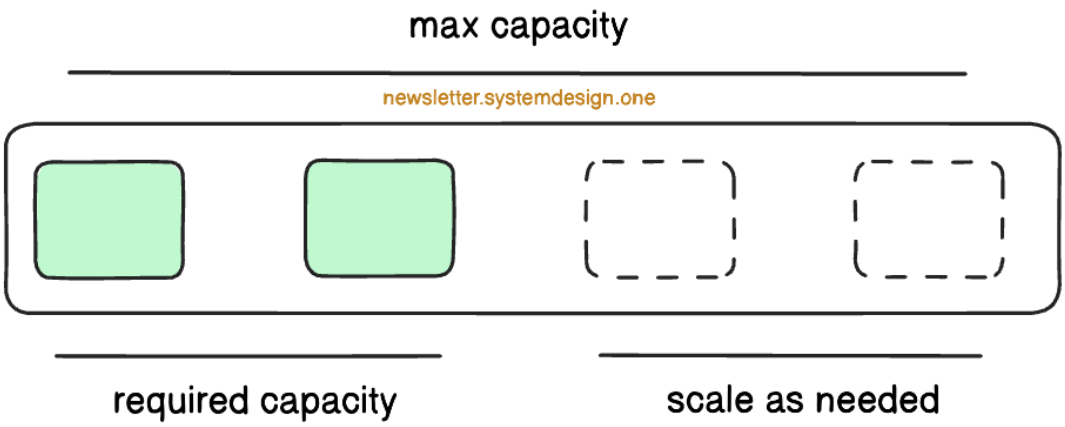
Users may see degraded results, but the rest of the system stays healthy. Once the failing part recovers, the breaker resets.



Circuit-breaker state machine

Autoscaling adds or removes resources based on demand (without an engineer's intervention)...

If traffic rises, new servers or containers start; if traffic falls, extra capacity shuts down. This keeps performance steady while saving cost. Sudden bursts get absorbed without breaking the system. A system able to grow and shrink based on usage is called an elastic system.



Autoscaling Explained

Health checks continuously watch each node's status...

Slow, failing, or unresponsive components trigger recovery before users feel the impact. Simple checks might only ping a service. Deeper checks verify functionality like *"can the database run a query?"*

Availability/load testing is the rehearsal before the outage...

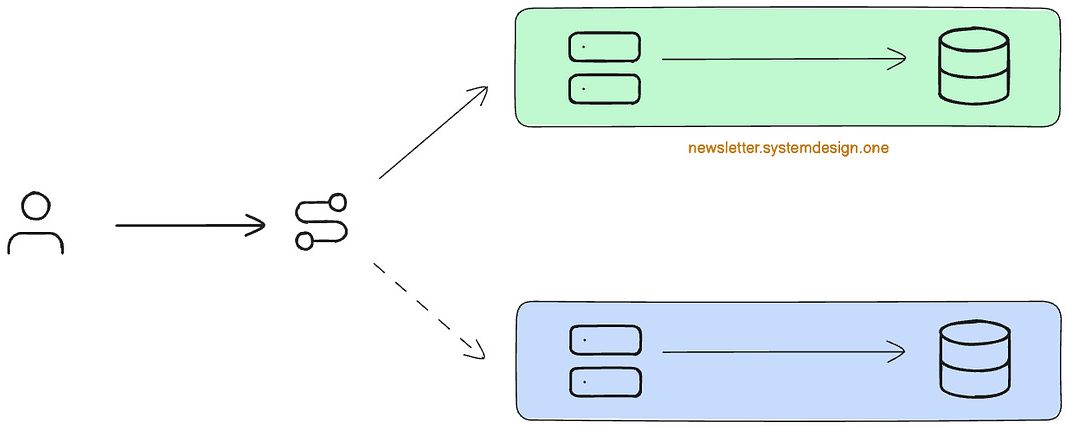
These tests simulate heavy traffic and push the system to the breaking point. Other experiments kill nodes or inject failures. These reveal weaknesses in the system's design and recovery process. Without testing, you only find problems in production when it's too late. With testing, engineers can patch weaknesses and gain confidence before real failures occur.

Deployment practices like blue/green and canary releases reduce the risk of updates...

In blue/green, two environments run side by side. You switch traffic to the new one once it's verified.

In Canary releases, only a small group of users sees the change first. If problems appear, rollback is quick and low-impact. Both approaches

ensure that new code doesn't take down the entire system.



Blue-green deployment

These techniques make failures mostly invisible to clients.

They protect the system, reduce stress, and keep core services running. Advanced techniques add protective layers, making everything smooth.

Operational Practices and Tradeoffs

Operations keep systems running when traffic spikes or parts fail.

Here are clear practices to stop small problems from becoming big outages.

Change management focuses on ensuring code changes don't break the existing system.

- Deploy updates in small steps
- Use feature flags or canary releases to test the change

- Ensure you can roll back fast if things break
- Track key metrics after the rollout to catch issues fast
- *Slow is smooth, smooth is fast. Move fast & break things is a bad HA strategy.*

Capacity planning ensures the system can handle traffic without hitting limits.

- Check traffic and server usage during normal operation (CPU, memory, network bandwidth, etc.)
- Add servers or resources before you hit the limits
- Keep a safety buffer so nothing runs at full capacity
- *Unused resources cost a fraction of what gets lost during downtime.*

Tradeoffs are choices engineers make to balance availability, cost, performance, and complexity. Every decision comes with a cost or limitation:

- **Costs vs Risk**: Many HA techniques introduce redundancy, which increases operational costs
- **Consistency vs Availability:** HA often comes at the cost of consistency, which brings a new suite of challenges
- **Complexity vs Simplicity:** Many different HA techniques combined make the system more complex, which makes it harder to maintain

These tradeoffs guide design decisions. They help engineers focus on what matters while keeping the system maintainable and affordable.

A high-availability system that is too complex or too costly is useless.

The bottom line

High availability is about choices, not just backups…

It's about using patterns, techniques, metrics, and operations to improve the system. Every choice is a tradeoff. The goal is a system that keeps running, handles traffic spikes, and serves users reliably. It must stay simple to maintain and affordable for the business.

Engineering for high availability is not about perfection...It's about finding the right balance that works in the long run.

Know someone who wants to learn system design? Consider gifting a subscription:

Give a gift subscription

👋 Find me on LinkedIn | Twitter | Threads | Instagram

Thank you for supporting this newsletter.

You are now 210,001+ readers strong, very close to 210k. Let's try to get 211k readers by 5 March. Consider sharing this post with your friends and get rewards.

Y'all are the best.

References

- Service Level Objectives - Google SRE Book
- High Availability and Disaster Recovery - Microsoft System Center
- Well-Architected Framework: Reliability - Google Cloud
- High Availability Architecture - Couchbase
- Establishing RPO and RTO Targets for Cloud Applications - AWS
- Synchronous and asynchronous database replication explained - Cockroach Labs
- Load Testing - Engineering Fundamentals Playbook Microsoft
- What is blue-green deployment? - RedHat
- Circuit-breaker pattern - Microsoft